

RDS — State Machine & Role Definition

Drone Remote Weapon Station — Aleppo Systems International Document for Manager Review & Approval Date: June 2026 **Status:** Pending Approval

Purpose

This document defines the **exact responsibilities** of each software component in the RDS system: - **ISM** — Independent Safety Monitor (STM32H7, High Integrity) - **FCC** — Fire Control Computer (NVIDIA Jetson, Medium Integrity) - **Operator Panel / GCS** — Ground Control Station WebApp (Low Integrity)

The goal is to establish a clear, agreed division of roles before further development, in compliance with MIL-STD-882E safety requirements.

1. System State Machine

1.1 States

Power-up	Any fault, watchdog,
Reset	comm loss, invalid cmd
	seq error, internal fault

SAFE	FAULT
------	-------

STANDBY cmd	Any violation
-------------	---------------

STANDBY

ARM cmd

MOTION commands
 Actuators (torque ON)
 ARMED

 DISARM / fault

 FIRE REQUEST
 + dual-channel
 interlock auth

 FIRING fault / fire complete

 FAULT RECOVERY RESET + authority SAFE

1.2 State Definitions

State	Torque	Fire	Description
SAFE	OFF	INHIBITED	Default safe state. Power-up, reset, any fault.
STANDBY	OFF	INHIBITED	System ready. Operator has acknowledged. Torque still off.
ARMED	ON	INHIBITED	Motion permitted. Torque enabled. Fire still inhibited.
FIRING	ON	ENABLED	Dual-channel interlock satisfied. Fire authorized.
FAULT	OFF	INHIBITED	Safety violation detected. Requires authority to recover.

2. ISM — Independent Safety Monitor

Processor: STM32H7 (dedicated MCU) **Runtime:** Bare-metal, no OS, 1ms hardware timer tick **Integrity Level:** High (MIL-STD-882E) **Communication:** CAN bus only — no Ethernet, no USB, no network

2.1 What ISM Does

#	Responsibility	How	Trigger
1	Enter SAFE on power-up	Assert torque-disable GPIO LOW at T+5ms	Power applied
2	Monitor FCC heartbeat	Software watchdog, 500ms timeout	Every 100ms heartbeat expected
3	Enter FAULT on comm loss	If no heartbeat within 500ms → FAULT	Watchdog expiry
4	Validate STANDBY request	Accept only from SAFE state	FCC CAN 0x103
5	Validate ARM request	Accept only from STANDBY state	FCC CAN 0x101
6	Enable torque on ARM	Assert torque-enable CAN frame 0x302 + GPIO	ARM accepted

#	Responsibility	How	Trigger
7	Validate motion commands	Check: range ($\pm 180^\circ$ az, $\pm 90^\circ$ el), rate (≤ 45 dps), direction, current state	FCC CAN 0x110
8	Reject invalid commands	→ FAULT + log	Any violation
9	Gate fire authorization	Require dual-channel interlock (ch-A AND ch-B)	FCC CAN 0x120
10	Disable torque on SAFE/FAULT	Deassert GPIO + CAN 0x302 immediately	Any fault / DISARM
11	Log all faults	Timestamped log: time, state, source, reason	Every fault
12	Enforce recovery procedure	Accept RECOVERY only with authority token, only from FAULT	FCC CAN 0x130
13	Hardware watchdog kick	Kick IWDG every 1ms tick	Always

2.2 What ISM Does NOT Do

X	Reason
Does NOT read encoder position	That is FCC's job — not safety-critical
Does NOT compute trajectory or PID	Medium integrity function — belongs in FCC
Does NOT communicate with GCS/WebApp	Non-safety software must not reach ISM
Does NOT stream video	Non-safety function
Does NOT run on Linux / Jetson	Linux is non-deterministic, not certifiable
Does NOT accept commands from WebApp directly	All commands must pass through FCC

2.3 ISM CAN Messages

Direction	CAN ID	Message	Rate
FCC → ISM	0x100	Heartbeat	10 Hz
FCC → ISM	0x101	ARM	On demand
FCC → ISM	0x102	DISARM	On demand
FCC → ISM	0x103	STANDBY	On demand
FCC → ISM	0x110	Motion command (az/el/rate)	10 Hz when ARMED
FCC → ISM	0x120	Fire request	On demand
FCC → ISM	0x130	Recovery reset	On demand
ISM → FCC	0x200	State ACK	On state change
ISM → FCC	0x210	Fault report	On fault
ISM → FCC	0x211	Heartbeat ACK	1 Hz
ISM → Actuators	0x302	Torque enable/disable	On state change
ISM → Actuators	0x310	Fire enable/inhibit	On state change

3. FCC — Fire Control Computer

Processor: NVIDIA Jetson Orin NX **Runtime:** Ubuntu Linux (PRE-EMPT_RT kernel) **Integrity Level:** Medium (MIL-STD-882E) **Communication:** CAN bus (to ISM), Ethernet (to GCS, to Micro300)

3.1 What FCC Does

#	Responsibility	How	Trigger
1	Send heartbeat to ISM	CAN 0x100 every 100ms	Timer thread
2	Receive ISM state	Parse CAN 0x200 STATE_ACK	On state change
3	Forward operator commands to ISM	Translate HTTP → CAN	WebApp HTTP POST
4	Read motor encoder position	CAN 0x300 (az/el feed- back)	10 Hz
5	Compute motion command	PID / tra- jectory from encoder + target	10 Hz when ARMED
6	Send motion command to ISM	CAN 0x110 (ISM vali- dates)	10 Hz when ARMED
7	Receive Micro300 video stream	H.265 RTSP over Ethernet	Continuous
8	Control gimbal	MavLink UDP to Micro300	On demand
9	Forward video + telemetry to GCS	UDP/JSON + RTP stream	Continuous
10	Serve HTTP API to WebApp	REST API port 5300	On request
11	Log system events	Timestamped log file	On event

3.2 What FCC Does NOT Do

X	Reason
Does NOT directly energize actuators	ISM gates all torque — SWR-SAFE-002
Does NOT enforce safety limits	That is ISM's job
Does NOT bypass ISM	All actuator commands must be validated by ISM
Does NOT run the watchdog	ISM has its own independent watchdog

3.3 FCC Threads

Thread	Function	Rate
heartbeatLoop	Send CAN HEARTBEAT to ISM	10 Hz
canReceiveLoop	Receive STATE_ACK / FAULT_REPORT from ISM	Continuous
motorCmdLoop	Read encoder → compute → send MOTION to ISM	10 Hz when ARMED
gcsPollLoop	Receive UDP commands from GCS	Continuous
HttpServer	Serve REST API to WebApp	On request
CameraStream	Receive H.265 from Micro300	30 fps

4. Operator Panel / GCS WebApp

Platform: Browser (Blazor WebApp) on GCS laptop **Runtime:** .NET / Browser **Integrity Level:** Low (MIL-STD-882E) **Communication:** HTTP to FCC REST API (port 5300)

4.1 What the Panel Does

#	Responsibility	How
1	Display ISM state	Poll GET /api/status every 1s
2	Display az/el / torque / camera status	Same status poll
3	Send STANDBY command	POST /api/standby → FCC → ISM
4	Send ARM command	POST /api/arm → FCC → ISM
5	Send DISARM / Emergency SAFE	POST /api/disarm → FCC → ISM
6	Send motion target	POST /api/move → FCC → Motor → ISM validate
7	Send gimbal commands	POST /api/gimbal → FCC → Micro300 MavLink
8	Send zoom commands	POST /api/zoom_vis
9	Request fire	POST /api/fire → FCC → ISM (dual-channel interlock)
10	Request recovery	POST /api/recovery → FCC → ISM

4.2 What the Panel Does NOT Do

X	Reason
Does NOT talk to ISM directly	ISM is safety-critical — all commands via FCC
Does NOT talk to Micro300 directly	Camera interface is FCC's responsibility

X	Reason
Does NOT enforce safety	Safety is ISM's job, not GUI's
Does NOT assume ISM state	Always reads from FCC status — never infers

5. Command Flow Summary

OPERATOR	FCC	ISM	ACTUATORS
POST /api/standby	CAN 0x103 STANDBY CAN 0x200 ACK		
{"result":"ok"}			
POST /api/arm	CAN 0x101 ARM CAN 0x200 ACK		
{"result":"ok"}		CAN 0x302	torque ON
POST /api/move az=45, el=20	[encoder read 0x300] CAN 0x110 MOTION		
	CAN 0x200 ACK [PID compute]	validates	
		move	

6. Open Questions for Manager Approval

The following items require manager decision before implementation:

#	Question	Options
1	Encoder CAN ID — which ID does the motor driver use?	0x300 (proposed) or per motor driver spec
2	Fire dual-channel — who provides channel B token?	Second operator? Hardware interlock?
3	Recovery authority — who is authorized to reset FAULT?	Operator only? Supervisor required?

#	Question	Options
4	Video forwarding — does FCC forward raw H.265 or decoded frames to GCS?	Raw (lower latency) or decoded
5	MavLink version — Micro300 MavLink version?	v1 or v2

*Aleppo Systems International — RDS Software Team — June 2026
Pending Manager Approval*